

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-4: Formulate data-driven web solutions using XML, XSLT, and JSP within the MVC framework. (L4)

Assessment Tool : Pseudocode Writing Task (Algorithm Design)
Weightage: 20%

QUESTIONS:

Total Marks: 25

1: XML Parsing Algorithm

Write a detailed pseudocode algorithm to parse an XML document that contains structured data such as user or product details. Your algorithm should clearly describe the steps involved in loading the XML file, accessing its elements, traversing through nodes, extracting required data, and displaying the output in a readable format. Ensure that the logic handles hierarchical structure properly.

1. XML Parsing Algorithm (Hierarchical Tree Traversal)

ALGORITHM: ParseAndExtractXML

INPUT: xmlFilePath (String)

OUTPUT: Structured console/report output of entities and nested attributes

BEGIN

 // Step 1: Initialize parser and load document

 TRY

 DECLARE parserBuilder AS DocumentBuilderFactory.newInstance()

 SET parserBuilder.setIgnoringElementContentWhitespace = TRUE

 DECLARE documentBuilder AS parserBuilder.newDocumentBuilder()

 DECLARE xmlDocument AS documentBuilder.parse(xmlFilePath)

 // Normalize document structure to eliminate redundant whitespace/text nodes

 CALL xmlDocument.getDocumentElement().normalize()

```

DECLARE rootNode AS xmlDocument.getDocumentElement()
PRINT "Root Element: " + rootNode.getNodeName()

// Step 2: Retrieve target entity nodes (e.g., <product> or <user>)
DECLARE entityList AS rootNode.getChildNodes()

// Step 3: Iterate through top-level entity nodes
FOR index FROM 0 TO entityList.getLength() - 1 DO
    DECLARE currentNode AS entityList.item(index)

    // Filter out comment or whitespace text nodes
    IF currentNode.getNodeType() EQUALS ELEMENT_NODE THEN
        PRINT "-----"
        PRINT "Entity: " + currentNode.getNodeName()

        // Extract element attributes (e.g., id="P101")
        IF currentNode.hasAttributes() THEN
            DECLARE attributes AS currentNode.getAttributes()
            FOR attrIdx FROM 0 TO attributes.getLength() - 1 DO
                DECLARE attr AS attributes.item(attrIdx)
                PRINT " [Attribute] " + attr.getNodeName() + " = " + attr.getNodeValue()
            END FOR
        END IF

        // Step 4: Recursively / sequentially extract child elements
        CALL TraverseAndPrintChildren(currentNode, 1)
    END IF
END FOR

CATCH FileNotFoundException THEN
    PRINT "Error: Specified XML file not found at path: " + xmlFilePath
CATCH XMLParseException AS ex THEN
    PRINT "Error: Malformed XML document. " + ex.getMessage()
END TRY
END

FUNCTION TraverseAndPrintChildren(parentNode, depthLevel)
    DECLARE childNodes AS parentNode.getChildNodes()
    DECLARE indent AS RepeatString(" ", depthLevel)

    FOR i FROM 0 TO childNodes.getLength() - 1 DO
        DECLARE child AS childNodes.item(i)

        IF child.getNodeType() EQUALS ELEMENT_NODE THEN
            // Check if child has deeper sub-elements or direct text
            IF HasChildElements(child) THEN
                PRINT indent + child.getNodeName() + ":"
                CALL TraverseAndPrintChildren(child, depthLevel + 1)
            ELSE
                PRINT indent + child.getNodeName() + ": " + child.getTextContent().trim()
            END IF
        END IF
    END FOR
END FUNCTION

```

```
END IF
END IF
END FOR
END FUNCTION
```

2: MVC Workflow

Write a pseudocode representation of the MVC (Model–View–Controller) workflow in a web application. Your answer should clearly illustrate how a user request is received, processed by the Controller, how the Model interacts with the database, and how the View generates and returns the response to the user. Include all major steps involved in request handling and response generation.

MVC Workflow Algorithm

ALGORITHM: MVC_Web_Request_Lifecycle

INPUT: Incoming HTTP Request Packet (URL, Query/Body, Headers, Cookies)

OUTPUT: HTTP Response Packet (Rendered View HTML / JSON payload)

BEGIN

// Step 1: Front Controller / Router intercepts request

DECLARE request AS ParseIncomingHttpRequest()

DECLARE response AS InitializeHttpResponse()

DECLARE targetRoute AS Router.resolve(request.getMethod(), request.getPath())

IF targetRoute IS NULL THEN

 response.setStatusCode(404)

 response.setBody("Page Not Found")

 RETURN response

END IF

// Step 2: Controller Execution

DECLARE controllerInstance AS Instantiate(targetRoute.getController())

DECLARE actionMethod AS targetRoute.getAction()

// Extract sanitized input data

DECLARE inputParams AS Sanitize(request.getParams())

// Step 3: Model Interaction & Business Logic

TRY

 DECLARE modelService AS Instantiate(ProductService)

 DECLARE domainData AS NULL

IF request.getMethod() EQUALS "POST" THEN

 // State-altering action

 DECLARE newEntity AS modelService.create(inputParams)

 domainData = modelService.persistToDatabase(newEntity)

ELSE

```

        // Read-only query action
        domainData = modelService.fetchData(inputParams.get("id"))
    END IF

    // Step 4: View Preparation
    DECLARE viewModel AS New Map()
    viewModel.put("data", domainData)
    viewModel.put("csrfToken", request.getSession().get("csrf_token"))

    // Step 5: View Rendering
    DECLARE viewTemplate AS LoadTemplate(targetRoute.getViewPath())
    DECLARE renderedContent AS viewTemplate.merge(viewModel)

    // Step 6: Return Response
    response.setStatusCode(200)
    response.setHeader("Content-Type", "text/html; charset=UTF-8")
    response.setBody(renderedContent)

    CATCH ValidationException AS valErr THEN
        response.setStatusCode(422)
        response.setBody(RenderView("errors/validation.html", valErr.getErrors()))
    CATCH DatabaseException AS dbErr THEN
        response.setStatusCode(500)
        response.setBody(RenderView("errors/500.html", "Database error occurred."))
    END TRY

    RETURN response
END

```

3: JSP Request Flow

Write a detailed pseudocode algorithm to explain the request–response flow in a JSP-based web application. Your algorithm should include steps such as client request initiation, server processing, interaction with backend components (like Servlets or database), and generation of dynamic content that is sent back to the client.

JSP Request–Response Processing Flow

ALGORITHM: JSP_Execution_Lifecycle

INPUT: Client HTTP GET/POST Request for target `.jsp` file

OUTPUT: Dynamic HTML payload streamed to client browser

BEGIN

// Step 1: Web Container intercepts request

DECLARE requestUri AS request.getRequestURI() // e.g., "/products.jsp"

// Step 2: Translation and Compilation Phase (Handled by Servlet/JSP Engine)

DECLARE servletClassFile AS FindCompiledServletClass(requestUri)

```

IF servletClassFile DOES NOT EXIST OR FileModifiedTime(requestUri) >
FileModifiedTime(servletClassFile) THEN
    // Parse JSP syntax into a Java Servlet source file (.java)
    DECLARE javaSourceCode AS JSPTranslator.translateToServlet(requestUri)

    // Compile Java source to Bytecode (.class)
    servletClassFile = JavaCompiler.compile(javaSourceCode)

    // Load class and invoke jspInit()
    DECLARE jspServletInstance AS ClassLoader.load(servletClassFile).newInstance()
    CALL jspServletInstance.jspInit()
ELSE
    DECLARE jspServletInstance AS GetLoadedServletInstance(servletClassFile)
END IF

// Step 3: Execution Phase (_jspService method invocation)
// Container assigns request to an execution thread
TRY
    DECLARE outputStream AS response.getWriter()

    // Implicit JSP Objects instantiated by container
    DECLARE pageContext AS CreatePageContext(request, response)
    DECLARE session AS request.getSession()
    DECLARE application AS request.getServletContext()

    // Execute business logic embedded or delegated by tags/beans
    DECLARE productDao AS New ProductDAO()
    DECLARE productList AS productDao.getAllProducts()

    // Dynamic HTML markup construction
    outputStream.write("<html><head><title>Product Catalog</title></head><body>")
    outputStream.write("<h1>Available Products</h1><ul>")

    FOR EACH product IN productList DO
        outputStream.write("<li>" + product.getName() + " - $" + product.getPrice() + "</li>")
    END FOR

    outputStream.write("</ul></body></html>")

    // Step 4: Finalize and Flush Stream
    response.setStatus(200)
    response.setContentType("text/html")
    outputStream.flush()

CATCH Exception AS ex THEN
    CALL HandleJspErrorPage(ex, request, response)
END TRY
END

```

4: Database Retrieval

Write a pseudocode algorithm to retrieve data from a database. Your answer should include steps for establishing a database connection, executing SQL queries, processing the result set, and displaying the retrieved data. Clearly represent the sequence of operations involved in database interaction.

Database Retrieval Algorithm (JDBC Sequence)

ALGORITHM: DatabaseDataRetrieval

INPUT: searchCriteria (String), dbUrl, dbUser, dbPassword

OUTPUT: List of Domain Objects displayed to user interface

BEGIN

 // Step 1: Declare resource handles

 DECLARE dbConnection AS NULL

 DECLARE preparedStatement AS NULL

 DECLARE resultSet AS NULL

 DECLARE productList AS New List()

TRY

 // Step 2: Load Driver & Establish Connection

 CALL Class.forName("com.mysql.cj.jdbc.Driver")

 dbConnection = DriverManager.getConnection(dbUrl, dbUser, dbPassword)

 // Step 3: Prepare Parameterized SQL Query (Prevents SQL Injection)

 DECLARE sqlQuery AS "SELECT id, name, price, stock_qty FROM inventory WHERE category = ? AND status = 'ACTIVE'"

 preparedStatement = dbConnection.prepareStatement(sqlQuery)

 // Bind dynamic parameter values

 preparedStatement.setString(1, searchCriteria)

 // Step 4: Execute SQL Query

 resultSet = preparedStatement.executeQuery()

 // Step 5: Process and Map Result Set Cursor

 WHILE resultSet.next() DO

 // Read column data using explicit data types

 DECLARE prodId AS resultSet.getInt("id")

 DECLARE prodName AS resultSet.getString("name")

 DECLARE prodPrice AS resultSet.getDouble("price")

 DECLARE prodStock AS resultSet.getInt("stock_qty")

 // Map raw row data to Entity Object

 DECLARE productObj AS New Product(prodId, prodName, prodPrice, prodStock)

 productList.add(productObj)

 END WHILE

 // Step 6: Render / Output Results

 IF productList.isEmpty() THEN

```

        PRINT "No records found matching criteria: " + searchCriteria
    ELSE
        PRINT "===== Query Results (" + productList.size() + " items) ====="
        FOR EACH item IN productList DO
            PRINT "ID: " + item.id + " | Name: " + item.name + " | Price: $" + item.price + " |
Stock: " + item.stock
        END FOR
    END IF

    CATCH SQLException AS sqlEx THEN
        PRINT "Database Error: " + sqlEx.getMessage()
        PRINT "SQLState Code: " + sqlEx.getSQLState()
    FINALLY
        // Step 7: Explicit Resource Cleanup in reverse order of creation
        IF resultSet IS NOT NULL THEN CALL resultSet.close()
        IF preparedStatement IS NOT NULL THEN CALL preparedStatement.close()
        IF dbConnection IS NOT NULL THEN CALL dbConnection.close()
    END TRY
END

```

5: PHP Validation

Write a pseudocode algorithm for validating user input in a web form using PHP concepts. The algorithm should include checks for empty fields, validation of input formats (such as email and password), handling invalid inputs with appropriate error messages, and allowing submission only when all validations are satisfied.

PHP Input Form Validation Algorithm

ALGORITHM: ProcessAndValidateForm

INPUT: HTTP \$_POST Superglobal array

OUTPUT: Validation Error Collection OR Success Redirection

BEGIN

// Step 1: Initialize error tracking structure and data variables

DECLARE errors AS New KeyValueCollection()

DECLARE sanitizedData AS New KeyValueCollection()

// Step 2: Verify request method

IF \$_SERVER["REQUEST_METHOD"] NOT EQUALS "POST" THEN

REDIRECT to "form.html"

TERMINATE

END IF

// Step 3: Validate and Sanitize Username / Full Name

DECLARE rawName AS \$_POST["fullName"]

IF rawName IS NULL OR Trim(rawName) EQUALS "" THEN

errors.put("fullName", "Full name is required.")

ELSE

```

        sanitizedData.put("fullName", HtmlSpecialChars(StripTags(Trim(rawName))))
    IF Length(sanitizedData.get("fullName")) < 3 THEN
        errors.put("fullName", "Full name must be at least 3 characters long.")
    END IF
END IF

// Step 4: Validate and Sanitize Email
DECLARE rawEmail AS $_POST["email"]
IF rawEmail IS NULL OR Trim(rawEmail) EQUALS "" THEN
    errors.put("email", "Email address is required.")
ELSE
    DECLARE          sanitizedEmail          AS          FilterVar(Trim(rawEmail),
FILTER_SANITIZE_EMAIL)
    IF NOT FilterVar(sanitizedEmail, FILTER_VALIDATE_EMAIL) THEN
        errors.put("email", "Invalid email format.")
    ELSE
        sanitizedData.put("email", sanitizedEmail)
    END IF
END IF

// Step 5: Validate Password Strength
DECLARE rawPassword AS $_POST["password"]
DECLARE rawConfirmPassword AS $_POST["confirmPassword"]

IF rawPassword IS NULL OR rawPassword EQUALS "" THEN
    errors.put("password", "Password is required.")
ELSE
    // Criteria: Min 8 chars, 1 uppercase, 1 lowercase, 1 number, 1 special char
    DECLARE passwordRegex AS "^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$"

    IF NOT RegexMatch(passwordRegex, rawPassword) THEN
        errors.put("password", "Password must be at least 8 characters long and include
uppercase, lowercase, digit, and special character.")
    ELSE IF rawPassword NOT EQUALS rawConfirmPassword THEN
        errors.put("confirmPassword", "Passwords do not match.")
    END IF
END IF

// Step 6: Check Validation Status
IF errors.size() > 0 THEN
    // Return errors to the presentation view with retained old input
    SET_HTTP_STATUS(422) // Unprocessable Entity
    RENDER_VIEW("register_form.php", { "errors": errors, "oldInput": sanitizedData })
ELSE
    // Step 7: Proceed with processing (Secure Password Hashing & Persistence)
    DECLARE hashedPassword AS PasswordHash(rawPassword, PASSWORD_BCRYPT)
    sanitizedData.put("password", hashedPassword)

    DECLARE isSaved AS DatabaseService.saveUser(sanitizedData)

```

```

IF isSaved THEN
    REDIRECT to "dashboard.php?status=success"
ELSE
    errors.put("general", "Internal server error. Please try again later.")
    RENDER_VIEW("register_form.php", { "errors": errors })
END IF
END IF
END

```

Code of Conduct:

/ T. Nikhil Kumar Reddy (192372024) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: T. Nikhil Kumar Reddy

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation

Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient